DPA-Style Attacks on HQC

Zhuo Huang[1], Weijia Wang[2,3](✉), Xiaogang Zhou[4] and Yu Yu[1,5](✉)

[1] Shanghai Jiao Tong University, Shanghai, China,
sh1kaku@sjtu.edu.cn,yuyu@yuyu.hk
[2] School of Cyber Science and Technology, Shandong University, Qingdao, China,
wjwang@sdu.edu.cn
[3] State Key Laboratory of Cryptography and Digital Economy Security, Shandong University, Qingdao, China
[4] China Telecom Quantum Information Technology Group Co., Ltd,
zhouxiaogang@chinatelecom.cn
[5] Shanghai Qi Zhi Institute, Shanghai, China

Abstract. HQC (Hamming Quasi-Cyclic) was selected as the fifth algorithm in the NIST suite of post-quantum cryptographic (PQC) standards. As the only code-based algorithm currently standardized by NIST, HQC offers a good balance between security assurance, performance, and implementation simplicity. Most existing power analyses against HQC are of the SPA style: they can recover secrets with a small number of traces, but can only tolerate limited noise. In this paper, we develop a chosen-ciphertext DPA-style attack methodology against HQC. We formalize a dedicated chosen-ciphertext setting in which the adversary selects $(\mathbf{u}, \mathbf{v})$ to target the intermediate value $\mathbf{v} \oplus (\mathbf{uy})$ over $\mathbb{F}_2[x]/(x^n - 1)$. We further optimize the attack by reducing its computational complexity and generalizing it to target masked HQC implementations. The proposed approach is validated through both simulation and practical experiments. In noiseless simulations, full-key recovery is achieved with just 10 traces, and the required number of traces increases linearly with 1/SNR. In practical evaluations on an STM32F4 microcontroller, the secret key can be recovered with 50 traces without profiling and 20 traces with profiling. When first-order masking is applied, key recovery on the same hardware target remains feasible by exploiting second-order features, requiring approximately 3,000 traces without profiling. Our results establish a direct and analyzable connection between leakage on $\mathbf{v} \oplus (\mathbf{uy})$ and end-to-end key recovery, emphasizing the necessity of higher-order masking countermeasures for HQC implementations.

Keywords: Side-channel attacks · Differential power analysis · HQC · Tap-based Toeplitz windowed projections · Masking countermeasures

1 Introduction

The advent of large-scale quantum computing poses a fundamental threat to the cryptographic assumptions underpinning nearly all modern public-key infrastructures. In response, the National Institute of Standards and Technology (NIST) launched an open PQC standardization process in 2017, culminating in draft standards for ML-KEM [Nat24b] and ML-DSA [Nat24a], with the hash-based SLH-DSA [Nat24c] as a conservative option and HQC [MAB+18] representing the code-based family due to its simplicity, efficiency, and strong security foundations. HQC [GAMA+25] offers a clean structure and efficient implementation, making it an attractive candidate for practical deployment as well as a natural target for implementation-level cryptanalysis. Understanding and mitigating potential side-channel leakages in HQC is therefore essential to ensure its secure adoption.

Side-channel attacks [Koc96] recover secrets by exploiting physical leakage from implementations. Among available modalities, power analysis [KJJ99] is central because switching activity in digital circuits produces measurable variations that are well captured by Hamming-weight or Hamming-distance models. Analyses are typically organized into profiling and non-profiling classes. Profiling [CRR02] attacks first train statistical models of the leakage on a reference device and later apply the learned model to traces from the target, often reducing the required number of traces but demanding labeled data and close device matching. Non-profiling attacks [KJJ99] instead use an analytical leakage assumption and test key-dependent hypotheses directly on the measurements; these methods offer broader portability across devices but generally require larger data volumes.

Simple Power Analysis (SPA) [MDS99] examines one or a few traces to read data-dependent structure at specific operations. A single trace may reveal intermediate values that leak partial information about the key, so recovering the full key can sometimes require multiple traces, although some works achieve full recovery from a single trace. It is interpretable and can succeed with very few measurements when leakage is substantial and timing is stable. Its main weakness is poor noise tolerance: amplitude fluctuations, desynchronization, random delays, and masking directly perturb the discriminant since there is little averaging to suppress perturbations. Differential Power Analysis (DPA) [KJJ99, BCO04, GBTP08] mitigates these issues by aggregating many traces and applying statistical hypothesis tests to separate key-dependent effects from noise. Averaging improves the signal-to-noise ratio, light alignment reduces residual timing variation, and correlation-based distinguishers built on Hamming-weight or Hamming-distance models provide robust ranking of hypotheses [BCO04]. DPA also extends to higher-order features that cancel random masks on average [Mes00]. The trade-off is a larger measurement cost, but the method is markedly more resilient to noise and variability across devices.

Masking countermeasures (see, e.g., [CJRR99, ISW03, NRR06, BBD+16, WMCS20, WJZY23] for an incomplete list of literature) protect sensitive intermediate values by splitting each value into several random shares and by ensuring that any single observable depends on at most one share. Cryptographic operations are executed on shares, and relinearization steps are used to produce correct masked outputs. Attacks against masked implementations, therefore, adopt higher-order strategies that combine several sample points [KP00, OMHT06]. After appropriate centering and combination [DZFL14], these higher-order statistics can reveal the masked secret despite the added randomness.

1.1 Contributions

In this paper, we study HQC decapsulation from a side-channel perspective and develop methods of DPA-style attacks that exploit the structure of the `vect_add` intermediates during decapsulation. Our contributions are summarized as follows.

- In HQC decryption, the ciphertext is $(\mathbf{u}, \mathbf{v})$ and the secret key is $\mathbf{y}$, where $\mathbf{v} \oplus (\mathbf{uy})$ is computed in $\mathbb{F}_2[x]/(x^n - 1)$ and then decoded by the concatenated code to recover the message. We propose the first DPA–style attack targeting this decryption algorithm. We formalize a chosen-ciphertext setting where the adversary selects $(\mathbf{u}, \mathbf{v})$ to target the intermediate $\mathbf{w} = \mathbf{v} \oplus (\mathbf{uy})$. Concretely, we design a structure for $\mathbf{u}$ such that each block of $\mathbf{w}$ only depends on a limited number of bits of $\mathbf{y}$, and we vary $\mathbf{v}$ to induce affine outputs. This construction yields short, high-rank linear projections of the secret $\mathbf{y}$ that are well suited to correlation against leakage of $\mathbf{w}$.

- We further optimize the attack method by improving its computational complexity and generalizing it to target the masked HQC scheme. We exploit the sparsity of $\mathbf{y}$ to propose an efficient key-guessing strategy that considers only candidate keys with limited Hamming weight. We also introduce a sliding-window variant that divides

each block into overlapping sub-blocks and reconstructs it via beam search on the overlaps. Furthermore, we present the first attack against masked HQC by targeting the centered second-order feature using the same ciphertext design.

- We validate the method in simulation and on hardware. In simulation, we sweep the noise standard deviation σ from 0 to 11 and observe that the required trace number follows $K \propto 1/\mathrm{SNR}$. In the noiseless case ($\sigma = 0$), full-key recovery is obtained with $K = 10$ traces, and at the highest noise tested ($\sigma = 11$) the sliding-window DPA achieves 100% recovery with $K = 65{,}000$ traces. For the practical attack, we acquire power traces from an STM32F4 target using ChipWhisperer, and perform all key-recovery computations offline. Our attack can recover the secret using 50 traces without profiling for unmasked case. When profiling is enabled, the number of required traces drops to 20. Against a first-order masked implementation on the same platform without profiling, our attack is able to recover the full secret key with $K = 3{,}000$ traces.

1.2 Related Work

Table 1: Representative power attacks on HQC implementations. Trace counts are normalized to noiseless or perfect-oracle settings.

Ref.	Attack type	Attack style	Op. targeted	#Traces	Mask eval.?
This work	Profiling & Non-profiling	DPA-style	`vect_add`	20	✔
[MNMD25]	Profiling	SPA-style	`expand_and_sum`	1	✗
[BMG+25]	Profiling	SPA-style	FHT	1	✗
[DG25a]	Profiling	SPA-style	Decoder_RM	2	✗
[DG25b]	Profiling	SPA-style	Decoder_RM	460	✗
[SHR+22]	Profiling	SPA-style	Decoder_RMRS	52,992	✗
[PRJ+25]	Profiling	SPA-style	Decoder_RM	1	✗
[GLG22]	Profiling	SPA-style	FHT	19,200	✗

Table 1 presents a synopsis of representative power-analysis attacks on HQC implementations. **Attack type** distinguishes profiling from non-profiling approaches. **Attack style** specifies SPA or DPA. **Op. targeted** identifies the HQC subroutine whose leakage is exploited. **Mask eval.?** indicates whether the work reports results on a masked implementation (which does not imply that other attacks cannot be extended to masked settings). **#Traces** reports the used trace numbers. We record the noiseless or perfect-oracle count when explicitly provided and the authors' empirical count otherwise. For profiled attacks, the reported **#Traces** refers to the number of traces used in the attack phase. Some listed SPA-style attacks require many traces because each trace only recovers partial key information, and multiple traces are needed to reconstruct the full key.

All works listed in Table 1 analyze power side-channel measurements. They primarily focus on decryption and extract leakage from the Reed–Muller decoder, the fast Hadamard transform (FHT), `expand_and_sum`, and in some cases the subsequent `find_peaks`. Maillet et al. [MNMD25] model `expand_and_sum` on a non-SIMD implementation. Paiva et al. [PRJ+25] drive the decoder with valid ciphertexts so that the protocol remains standard while the same Reed–Muller computations are executed, also enabling single-trace recovery with profiling. Dong et al. [DG25b] reduce the oracle budget for decoder-oriented profiling attacks to a few hundred queries through improved statistical analysis. In later work, they [DG25a] exploit decoder-side power leakage to instantiate multi-value plaintext-checking full-decryption oracle. Schamberger et al. [SHR+22] target the Reed–Solomon stage in the HQC-RMRS variant and report about fifty thousand traces for key recovery from decoding computations. Baisse et al. [BMG+25] construct a factor-graph model of the FHT and apply soft-analytical side-channel analysis with belief propagation on power

traces. Goy et al. [GLG22] exploit near-field electromagnetic leakage from the FHT, using a profiled multi-class classifier with soft-decision fusion over a few traces per bit to achieve practical full-key recovery.

Similar to most related works, our attack setting involves adversarially crafted ciphertexts and may therefore drive decapsulation into the failure branch, which is reminiscent of plaintext-checking oracle attacks. However, we do not rely on a plaintext-checking signal; instead we correlate analog leakage from the early XOR $\mathbf{w} = \mathbf{v} \oplus (\mathbf{uy})$ under chosen $(\mathbf{u}, \mathbf{v})$. Plaintext-checking oracle attacks are often SPA-style and concentrate most of the effort in offline modeling, making the online phase fast.

Although our DPA-style approach is less efficient and needs more traces in practice, since it relies on multi-trace statistics across many blocks. Concretely, we target the word-wise XOR at `vect_add` and use windowed Toeplitz projections so that each projection output depends only on a small, contiguous subset of secret bits, enabling hypothesis ranking with a statistical distinguisher in a divide-and-conquer manner. The trace complexity follows the usual noise scaling: N grows roughly proportional to the noise variance i.e. N $\propto 1/\mathrm{SNR}^2$. This complements prior works that focused on SPA-style leakage from decoder or FHT internals.

Timing and microarchitectural oracles form a complementary line that reduces proximity assumptions by exploiting platform effects. Examples include plaintext-checking oracles and division-induced variability under specific microarchitectures, often requiring thousands of oracle calls rather than analog traces [SGG24, GNNJ23]. These approaches are outside the scope of our power-trace focus and are therefore not listed in Table 1.

1.3 Organization of the Paper

Section 2 introduces notation, recalls the HQC PKE and KEM interfaces and the decapsulation. Section 3 presents the attack methodology based on Toeplitz windowed projections and details a DPA route with sliding windows and beam merging. Section 4 reports the experimental results, covering simulation and practical results on STM32F4 with ChipWhisperer in both unmasked and masked settings, and provides trace counts and success rates. In addition, we conduct a profiling DPA experiment to demonstrate that our design applies to both profiling and non-profiling adversaries. Empirically, profiling substantially reduces the required number of traces. The final Section 5 concludes this paper and offers future research suggestions.

2 Preliminary

2.1 Notations

We denote the binary finite field by $\mathbb{F}_2$, where addition and subtraction coincide. The main algebraic structure used in HQC is the polynomial ring $\mathcal{R} = \mathbb{F}_2[X]/(X^n - 1)$, where n is a positive integer. Throughout this paper, vectors are denoted by lowercase bold letters (e.g., $\mathbf{h}$). A polynomial in $\mathcal{R}$ can be equivalently represented as a binary vector of length n, and we use these two representations interchangeably. The Hamming weight of a vector $\mathbf{h}$, defined as the number of non-zero coefficients, is denoted by $w_H(\mathbf{h})$. For any vector $\mathbf{x}$, its support set $\mathrm{Supp}(\mathbf{x})$ is the set of indices corresponding to its non-zero entries.

Let $\mathbf{x} = (x_0, x_1, \ldots, x_{n-1}) \in \mathbb{F}_2^n$. The symbol x_i denotes its i-th component, with indices taken modulo n when convenient. In polynomial representation, these notations are equivalent to the coefficient subset of $x(X) = \sum_{i=0}^{n-1} x_i X^i \in \mathcal{R}$.

Given two vectors (or equivalently, two polynomials) $\mathbf{a}, \mathbf{b} \in \mathbb{F}_2^n$, their product in the

ring $\mathcal{R}$ is defined as a cyclic convolution:

$$\mathbf{c} = \mathbf{a} \cdot \mathbf{b} = (c_0, c_1, \ldots, c_{n-1}), \quad c_k = \bigoplus_{i+j \equiv k \pmod{n}} a_i b_j.$$

This corresponds to multiplying the polynomial representations $u(X) = \sum_{i=0}^{n-1} u_i X^i$ and $v(X) = \sum_{j=0}^{n-1} v_j X^j$ modulo $(X^n - 1)$, where the reduction enforces cyclicity among coefficients—an essential property of the quasi-cyclic structure of $\mathcal{R}$.

Block partitioning. For a set of vectors $\{\mathbf{x}_i | i = 1, \ldots, m\} \in \{\mathbb{F}_2^n\}^m$, we write the j-th *coordinate* of $\mathbf{x}_i$ as $(\mathbf{x}_i)_j$. For a fixed block length $\nu \geq 1$, each vector $\mathbf{x}_i$ can be partitioned into $s = \lceil n/\nu \rceil$ consecutive blocks and denoted as:

$$\mathbf{x}_i = \left(\mathbf{x}_i^{[0,\nu]}, \mathbf{x}_i^{[1,\nu]}, \ldots, \mathbf{x}_i^{[s-1,\nu]}\right), \quad \mathbf{x}_i^{[j,\nu]} \in \mathbb{F}_2^{\nu} \text{ for } j < s-1,$$

and if $\nu \nmid n$ then the last block $\mathbf{x}_i^{[s-1,\nu]}$ has length $n - (s-1)\nu$. We use the slice notation $\mathbf{x}_{[a:b]}$ for the contiguous subvector containing indices a through b. Each block $\mathbf{x}_i^{[j,\nu]}$ thus represents the slice $(\mathbf{x}_i)_{[j\nu:(j+1)\nu-1]}$. The k-th element within block j is denoted by $(\mathbf{x}_i^{[j,\nu]})_k$. Whenever ν divides n, the flat indexing relation holds:

$$(\mathbf{x}_i)_{j\nu+k} = (\mathbf{x}_i^{[j,\nu]})_k, \quad j \in \{0, \ldots, s-1\},\ k \in \{0, \ldots, \nu-1\}.$$

Notation summary. Throughout the paper, we use the following unified conventions:

- $\mathbf{x}_i$: the i-th vector in a family $\{\mathbf{x}_i\}$ satisfying $\mathbf{x}_i \in \mathbb{F}_2^n$;
- $\mathbf{x}_{[a:b]}$: the contiguous subvector containing indices a through b;
- $(\mathbf{x})_i$: the i-th coordinate of a vector $\mathbf{x}$;
- $\mathbf{x}^{[j,\nu]}$: the j-th block of $\mathbf{x}$ under block size ν;
- $(\mathbf{x}^{[j,\nu]})_k$: the k-th entry within block j under block size ν;
- $\mathbf{x}^{(i)}$: the i-th masked share of vector $\mathbf{x}$ under Boolean masking, such that $\mathbf{x} = \mathbf{x}^{(0)} \oplus \mathbf{x}^{(1)}$.

2.2 Hamming Quasi-Cyclic (HQC)

The Hamming Quasi-Cyclic (HQC) scheme is a code-based cryptosystem. Its security relies on the difficulty of decoding a general linear code. HQC is first designed as an IND-CPA secure Public Key Encryption (PKE) scheme, which is then converted into an IND-CCA secure Key Encapsulation Mechanism (KEM) using a variant of the Fujisaki–Okamoto (FO) transformation. The official HQC submission specifies three parameter sets for different security levels, as detailed in Table 2.

Table 2: Parameter sets for HQC as defined in the official specification [GAMA+25]

Instance	**Security**	n_1	n_2	n	k	ω	$\omega_r = \omega_e$	**DFR**
HQC-1	NIST-1	46	384	17 669	128	66	75	$< 2^{-128}$
HQC-3	NIST-3	56	640	35 851	192	100	114	$< 2^{-192}$
HQC-5	NIST-5	90	640	57 637	256	131	149	$< 2^{-256}$

2.2.1 The PKE Scheme

According to the specification of [GAMA+25], the PKE core of HQC consists of three main algorithms: key generation, encryption, and decryption. Uniform random sampling from a finite set S is denoted by $\xleftarrow{\$} S$, and sampling an element from $\mathcal{R}$ with a fixed Hamming weight w is denoted by $\xleftarrow{\$} \mathfrak{S}(\mathcal{R}, w)$.

Key Generation. To generate a key pair, a random polynomial $\mathbf{h} \xleftarrow{\$} \mathcal{R}$ is sampled. Two additional polynomials, $\mathbf{x} \xleftarrow{\$} \mathfrak{S}(\mathcal{R}, \omega)$ and $\mathbf{y} \xleftarrow{\$} \mathfrak{S}(\mathcal{R}, \omega)$, are sampled with a fixed Hamming weight ω. The public key is then defined as $\mathbf{pk} = (\mathbf{h}, \mathbf{s})$, where $\mathbf{s} = \mathbf{x} + \mathbf{hy}$. The corresponding secret key is $\mathbf{sk} = (\mathbf{x}, \mathbf{y})$.

Encryption. To encrypt a message $\mathbf{m}$, three error vectors are sampled: $\mathbf{r}_1 \xleftarrow{\$} \mathfrak{S}(\mathcal{R}, \omega_r)$, $\mathbf{r}_2 \xleftarrow{\$} \mathfrak{S}(\mathcal{R}, \omega_r)$, and $\mathbf{e} \xleftarrow{\$} \mathfrak{S}(\mathcal{R}, \omega_e)$, with fixed Hamming weights ω_r and ω_e. The message $\mathbf{m}$ is encoded into a codeword $\mathbf{mG}$ using a public generator matrix $\mathbf{G}$. The ciphertext $\mathbf{c} = (\mathbf{u}, \mathbf{v})$ is computed as:

$$\mathbf{u} = \mathbf{r}_1 \oplus \mathbf{hr}_2$$
$$\mathbf{v} = \mathbf{mG} \oplus \mathbf{sr}_2 \oplus \mathbf{e}$$

Decryption. Given a ciphertext $\mathbf{c} = (\mathbf{u}, \mathbf{v})$ and the secret key $\mathbf{sk} = (\mathbf{x}, \mathbf{y})$, the recipient first computes an intermediate value $\mathbf{w}$:

$$\mathbf{w} = \mathbf{v} \oplus (\mathbf{uy}) = (\mathbf{mG} \oplus \mathbf{sr}_2 \oplus \mathbf{e}) \oplus (\mathbf{r}_1 \oplus \mathbf{hr}_2)\mathbf{y} = \mathbf{mG} \oplus \mathbf{xr}_2 \oplus \mathbf{yr}_1 \oplus \mathbf{e}.$$

The term $\mathbf{e}' = \mathbf{xr}_2 \oplus \mathbf{yr}_1 \oplus \mathbf{e}$ is a noise vector with a low Hamming weight by construction. The recipient then uses a decoding algorithm, Decode($\mathbf{w}$), to correct the errors in $\mathbf{e}'$ and recover the original message $\mathbf{m}$. The decoding succeeds if $w_H(\mathbf{e}')$ is within the error-correction capability of the code.

2.2.2 The KEM Scheme

To achieve IND-CCA security, the PKE scheme is transformed into a KEM. The encapsulation and decapsulation procedures are illustrated in Figure 1.

KEM.Encaps($\mathbf{pk}$)	**KEM.Decaps**($\mathbf{c}, \mathbf{sk}$)
1: $\mathbf{m} \xleftarrow{\$} \{0,1\}^k$	1: $\mathbf{m}' \leftarrow$ PKE.Decrypt($\mathbf{sk}, \mathbf{c}$)
2: $\theta \leftarrow G(\mathbf{m} \Vert \mathbf{pk})$	2: $\theta' \leftarrow G(\mathbf{m}' \Vert \mathbf{pk})$
3: $\mathbf{c} \leftarrow$ PKE.Encrypt($\mathbf{pk}, \mathbf{m}; \theta$)	3: $\mathbf{c}' \leftarrow$ PKE.Encrypt($\mathbf{pk}, \mathbf{m}'; \theta'$)
4: $\mathbf{K} \leftarrow H(\mathbf{m}, \mathbf{c})$	4: if $\mathbf{c} = \mathbf{c}'$ and $\mathbf{m}' \neq \perp$ then
5: return $(\mathbf{c}, \mathbf{K})$	5: $\quad \mathbf{K} \leftarrow H(\mathbf{m}', \mathbf{c})$
	6: else
	7: $\quad \mathbf{K} \leftarrow H(\sigma, \mathbf{c})$ for a random σ
	8: end if
	9: return $\mathbf{K}$

Figure 1: The HQC Key Encapsulation and Decapsulation procedures.

3 Attack Methodology

We propose an implementation-driven methodology for key recovery from HQC decapsulation traces. At a high level, we use several Toeplitz polynomials to obtain localized linear projections of the secret and analyze them with a DPA framework. We used Pearson correlation in our experiments, but it can also be replaced by other statistical discriminators such as Mutual Information Analysis (MIA). An efficient sliding-window variant reduces enumeration cost, and the same method of selecting $\mathbf{u}$ is extended to first-order Boolean masking by using second-order features. Detailed constructions and parameters are given in the following subsections.

3.1 Attack Model

We consider an adversary with physical access who mounts a chosen-ciphertext attack and collects power traces during decapsulation. The adversary issues ciphertexts $\mathbf{u}$ and $\mathbf{v}$ to trigger the intermediate $\mathbf{w} = \mathbf{v} \oplus (\mathbf{uy})$, which is divided by ν-bit block. In each query, we sample the ciphertext component $\mathbf{v} \xleftarrow{\$} \mathcal{R}$, while choosing $\mathbf{u}$ according to our structured design.

Each DPA step targets a ν-bit block. In every block, the dedicated choosing polynomial $\mathbf{u}$ maps ν-bit $\mathbf{y}$ to an m-bit slice of the intermediate $\mathbf{w} = \mathbf{v} \oplus (\mathbf{uy})$. Each query yields a measurement correlated with the estimated leakage (e.g., Hamming weight) m-bit $\mathbf{w}$. By applying a statistical distinguisher to correlate predicted Hamming weights with the traces, we identify the ν-bit block $\mathbf{y}$. Repeating this step as needed recovers the secret $\mathbf{y}$.

The methodology applies to HQC-1, HQC-3, HQC-5 without change to the algebraic structure or polynomial $\mathbf{u}$ design; only lengths and weight parameters differ. For clarity and to match the hardware setup, the rest of this section presents HQC-1 and then carries over the same analysis to larger parameter sets.

3.2 Toeplitz-Based Local Projections for ν-Bit Blocks

Following the attack model in Section 3.1, our target is the Hamming-weight leakage arising from the intermediate $\mathbf{v} \oplus \mathbf{uy}$ during HQC decapsulation. We write $\mathbf{z} = \mathbf{u} \cdot \mathbf{y} \in \mathcal{R}$ and $\mathbf{w} = \mathbf{v} \oplus \mathbf{z}$. Our objective is blockwise recovery of $\mathbf{y}$. A single DPA analysis recovers a ν-bit block of $\mathbf{y}$, and a typical choice of ν is set to either 32 or 64. We construct several polynomials $\mathbf{u}(X)$ that enforce local dependence between a small number of selected output m-bit $\mathbf{z} = \mathbf{u} \cdot \mathbf{y}$ and the input ν-bit from $\mathbf{y}$ inside one single block. This locality enables the recovery of each block of $\mathbf{y}$ from the leakage corresponding to each m-bit block of $\mathbf{w}$.

We now describe how we construct the polynomial $\mathbf{u}$. Let T be a finite set of monomial indices chosen from $\{0, \ldots, \nu - m\}$. We choose the polynomial

$$\mathbf{u}(X) = \sum_{t \in T} X^{\,n-t} \in \mathcal{R},$$

which places ones in $\mathbf{u}$ at positions indexed by $n - t$ for each $t \in T$ and zeros elsewhere. For any output index r we have

$$z_r = \sum_{j=0}^{n-1} y_j \, u_{r-j \bmod n}.$$

Since $u_{r-j \bmod n} = 1$ holds exactly when $r - j \equiv n - t \pmod{n}$ for some $t \in T$, equivalently $j \equiv r + t \pmod{n}$, the sum reduces to the XOR over those positions. Hence

$$z_r = \bigoplus_{t \in T} y_{r+t \bmod n}.$$

Restrict the output index to $r \in \{s, \ldots, s+m-1\}$. Since the selected monomial indices $0 \le t \le \nu - m$, every shifted index remains inside the block:

$$s \le r \le s+m-1 \text{ and } 0 \le t \le \nu - m \implies s \le r+t \le s+m-1+\nu-m = s+\nu-1.$$

No wraparound occurs in this range, which means that each of the first m outputs depends only on the ν input bits inside the block:

$$z_s, z_{s+1}, \ldots, z_{s+m-1} \text{ are linear functions of } \mathbf{y}_{[s:s+\nu-1]}.$$

It is convenient to collect these m relations into a compact linear matrix. Introduce zero-based local indices.

$$r' := r - s \in \{0, \ldots, m-1\}, \qquad c' := c - s \in \{0, \ldots, \nu-1\}.$$

Define a binary matrix $\mathbf{U}^{(m\times\nu)}$ by

$$(\mathbf{U})_{r',c'} = 1 \iff c' = r' + t \text{ for some } t \in T, \qquad \text{and} \qquad (\mathbf{U})_{r',c'} = 0 \text{ otherwise.}$$

Then the first m outputs satisfy the banded Toeplitz relation

$$\mathbf{z}_{[s:s+m-1]} = \mathbf{U}^{(m\times\nu)}\, \mathbf{y}_{[s:s+\nu-1]}. \tag{1}$$

Each row of $\mathbf{U}^{(m\times\nu)}$ is obtained by shifting the previous row one position to the right, and all nonzero entries remain within the ν-bit block. Figure 2 illustrates this banded Toeplitz projection: each ν-bit block of $\mathbf{y}$ contributes to the lower m-bit of the corresponding block of $\mathbf{z}$. The rectangle on the left depicts $\mathbf{U}$ with constant diagonals, and the arrows indicate that the same $\mathbf{U}$ applies to each block of $\mathbf{y}$. The shaded segment on $\mathbf{z}$ highlights the first m output bits referenced in Equation 1. The labels $y[i]$ and $z[i]$ mark block indices along the block. The row shifts correspond to the selected monomial index in the set T, where smaller indices occupy the main diagonal and larger ones appear on upper diagonals. All nonzero entries are confined within the block columns, confirming that no wraparound occurs.

Example 1. This example shows how a small set of selected monomial indices defines a banded Toeplitz mapping from a contiguous block of $\mathbf{y}$ to a short block of outputs. Fix a block of length $\nu = 16$ starting at index s, and target $m = 8$ consecutive outputs beginning at the same position. The secret block is $\mathbf{y}_{[s:s+15]} \in \{0,1\}^{16}$. Let $T = \{0, 3, 6\} \subseteq \{0, \ldots, \nu - m\} = \{0, \ldots, 8\}$ be the set of selected monomial indices. Define the polynomial

$$\mathbf{u}(X) = \sum_{t\in T} X^{\,n-t} \in \mathcal{R},$$

so in this case $\mathbf{u}(X) = 1 + X^{\,n-3} + X^{\,n-6}$,

which places ones in $\mathbf{u}$ at delays n, $n-3$, and $n-6$ modulo n. By cyclic convolution, the coefficient at position r is

$$z_r \;=\; \bigoplus_{t\in\{0,3,6\}} y_{r+t \bmod n} \;=\; y_r \;\oplus\; y_{r+3 \bmod n} \;\oplus\; y_{r+6 \bmod n}.$$

Restricting to the first m outputs inside the block, write $r = s + r'$ with $r' \in \{0, \ldots, 7\}$. Because $T \subseteq \{0, \ldots, \nu - m\}$, each shifted index $r + t$ remains inside $[s, s + \nu - 1]$ and no wrap-around occurs. Hence for $r' = 0, \ldots, 7$,

$$z_{s+r'} \;=\; y_{s+r'} \;\oplus\; y_{s+r'+3} \;\oplus\; y_{s+r'+6}.$$

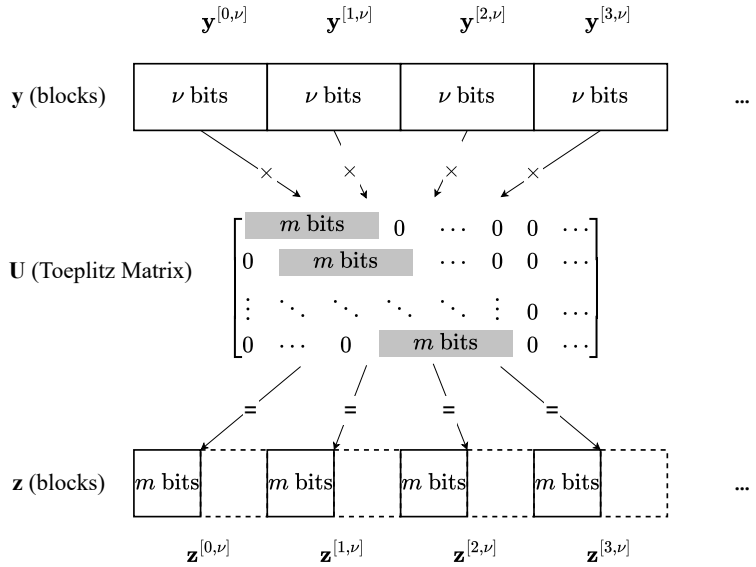


Figure 2: Banded Toeplitz projection $\mathbf{U}$ over a ν-bit block. Each ν-bit block of $\mathbf{y}$ maps to the lower m-bit of $\mathbf{z}$.

We now write the mapping explicitly as a binary matrix. Using local indices $r' = r - s \in \{0, \ldots, 7\}$ and $c' = c - s \in \{0, \ldots, 15\}$, we have $\mathbf{z}_{[s:s+7]} = \mathbf{U}^{(8\times16)}\, \mathbf{y}_{[s:s+15]}$ with

$$\mathbf{U}^{(8\times16)} = \begin{bmatrix} \boxed{1\ 0\ 0\ 1\ 0\ 0\ 1\ 0}\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ 0 \\ 0\ \boxed{1\ 0\ 0\ 1\ 0\ 0\ 1\ 0}\ 0\ 0\ 0\ 0\ 0\ 0\ 0 \\ 0\ 0\ \boxed{1\ 0\ 0\ 1\ 0\ 0\ 1\ 0}\ 0\ 0\ 0\ 0\ 0\ 0 \\ 0\ 0\ 0\ \boxed{1\ 0\ 0\ 1\ 0\ 0\ 1\ 0}\ 0\ 0\ 0\ 0\ 0 \\ 0\ 0\ 0\ 0\ \boxed{1\ 0\ 0\ 1\ 0\ 0\ 1\ 0}\ 0\ 0\ 0\ 0 \\ 0\ 0\ 0\ 0\ 0\ \boxed{1\ 0\ 0\ 1\ 0\ 0\ 1\ 0}\ 0\ 0\ 0 \\ 0\ 0\ 0\ 0\ 0\ 0\ \boxed{1\ 0\ 0\ 1\ 0\ 0\ 1\ 0}\ 0\ 0 \\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ \boxed{1\ 0\ 0\ 1\ 0\ 0\ 1\ 0}\ 0 \end{bmatrix}.$$

The matrix is banded Toeplitz: each row is a one-position right shift of the previous row, and all nonzero entries remain within the ν-bit block. The monomial corresponding to index 0 places ones on the main diagonal of the upper-left 8×8 submatrix, giving full row rank $m = 8$ over $\mathbb{F}_2$. This ensures that, in the absence of noise, the m output bits provide m independent linear constraints on the ν input bits. The additional monomials with indices 3 and 6 widen the nonzero band and increase the dynamic range of Hamming weights, which improves separability for correlation-based analysis.

We therefore adopt a block-oriented parameterization in which each ν-bit block of $\mathbf{y}$ is treated as an independent secret block of length $\nu = 64$. For the DPA-style analysis, the polynomial design projects this 64-bit block onto $m = 32$ consecutive outputs of $\mathbf{z}$ and $\mathbf{w} = \mathbf{v} \oplus \mathbf{z}$. This choice aligns with the common 32-bit processing granularity on embedded targets, where a 64-bit block is divided into two 32-bit halves, and the instantaneous power tends to follow the Hamming weight of the active half.

3.3 DPA-Style Attack

The DPA-style attack turns leakage into statistical evidence accumulated over many traces. The goal is to rank hypotheses for each ν-bit block of $\mathbf{y}$ according to how well their predicted intermediates explain the measured power. The attack uses the linear relation

in Eq. (1). For a fixed ciphertext index i, the model produces the intermediate $\mathbf{w}_i$. The Hamming weight of this intermediate forms the predictor for power consumption.

Throughout the paper, we use the term DPA-style to denote multi-trace statistical attacks that rank key hypotheses using a leakage model together with a statistical distinguisher. Although our experiments use the Pearson correlation coefficient and therefore implement a Correlation Power Analysis, the same methodology can also be instantiated with a MIA distinguisher. For this reason, we refer to our approach as a DPA-style attack.

Parameter configuration follows Section 3.2. The block length ν equals 64 and the output length m equals 32. Each ν-bit block of $\mathbf{y}$ contributes to the low m bits of the product and to the corresponding XOR intermediate $\mathbf{w}$. This matches how 32-bit microcontrollers process 64-bit lanes, and therefore, the leakage model is defined at 32-bit granularity. For the DPA-style attack, the Toeplitz matrices $\mathbf{U}_i$ are densely and randomly generated to maximize the avalanche effect. Stacking several projections increases rank, reduces the intersection of nullspaces, and improves hypothesis separability without breaking block locality.

The search is regularized by the sparsity structure of HQC-1. The secret $\mathbf{y}$ has Hamming weight equal to 66 over n equal to 17669, which corresponds to a per-bit density $p = 66/17669 \approx 3.735 \times 10^{-3}$. Treating bit locations as approximately independent Bernoulli trials yields a tight approximation for per-block statistics. Let W_j denote the weight of the j-th 64-bit block of secret $\mathbf{y}$. The probability that a single block has a weight of at most four is

$$\Pr[W_j \leq 4] = \sum_{i=0}^{4} \binom{64}{i} p^i (1-p)^{64-i} \approx 0.999995. \tag{2}$$

There are B equal to $\lfloor n/64 \rfloor$ full blocks in total, which gives B equal to 276 when the short tail is ignored. Assuming independence across blocks gives

$$\Pr\big[\forall j \in \{0, \ldots, B-1\},\, W_j \leq 4\big] \approx (0.999995)^{276} \approx 0.9987. \tag{3}$$

Similarly, the probability that a single 64-bit block weighs at most 3 is ≈ 0.999897; across the $B = 276$ whole 64-bit blocks, this yields ≈ 0.9719. For 32-bit partitioning, the probability that a single 32-bit block weighs at most 4 is ≈ 0.999999865; across the 552 complete 32-bit blocks, this yields ≈ 0.999926.

The search space is bounded by sparsity. The secret $\mathbf{y}$ has Hamming weight equal to 66 over n equal to 17669. A binomial approximation shows that a 64-bit block contains at most four ones with overwhelming probability, as in Eq. (2) and Eq. (3). Enumeration is therefore restricted to

$$\mathcal{C}_{64,\leq w_{\max}} = \{\mathbf{y}^{[c,64]} \in \{0,1\}^{64} \,:\, w_H(\mathbf{y}^{[c,64]}) \leq w_{\max}\}, \qquad w_{\max} = 4, \tag{4}$$

which preserves the actual block with very high probability while keeping the domain tractable. In particular, per-block enumeration size is $\sum_{i=0}^{4} \binom{64}{i} = 6.79 \times 10^5$, which is acceptable in a multi-process setting. For comparison, $|\mathcal{C}_{32,\leq 4}| = 4.14 \times 10^4$ and $|\mathcal{C}_{16,\leq 4}| = 2.52 \times 10^3$ across n=17669, this yields about 552 and 1104 blocks and roughly 2.3×10^7 and 2.8×10^6 tested hypotheses in total, respectively. Although smaller blocks reduce the enumeration cost, they tend to increase noise and weaken the distinguisher in practice. As noted above, using 64-bit blocks while mapping to m=32 aligns well with the 32-bit microcontroller architecture at the targeted word-wise operation and thus maximizes the exploitable side-channel leakage. Consequently, $\nu = 64$ and $m = 32$ provide a practical balance between implementation alignment, signal strength, and search complexity.

For non-profiling DPA, we proceed by direct hypothesis testing targeting the block. For each candidate block value, we compute the corresponding Hamming-weight sequence predicted across the chosen ciphertexts and then evaluate the Pearson correlation between

this sequence and the measured samples at every time index. The candidate score is the maximum absolute correlation over time, and the highest-scoring candidate is selected.

After aligning all traces to the XOR site of $\mathbf{w} = \mathbf{v} \oplus (\mathbf{uy})$, we compute the Pearson correlation between the predicted Hamming-weight sequence $\mathrm{HW}_i(\mathbf{y}')$ and the measured samples Tr_i at each leakage index k. The candidate's score is

$$\mathbf{score}[\mathbf{y}'] \leftarrow \max_k \left|\mathrm{corr}\left(\mathbf{h}^{[c,\nu]}, \mathbf{t}_k\right)\right|$$

that is, the maximum absolute Pearson correlation over time.

Algorithm 1 and Algorithm 2 summarize the workflow. The first ranks candidates for a single block by correlation, and the second aggregates blockwise results to recover the whole secret. We search over the entire capture window and score candidates by the maximum absolute Pearson correlation across all sampling indices.

Algorithm 1: DPA Subroutine for a Single Block

Input: K ciphertext pairs $(\mathbf{u}_i, \mathbf{v}_i)$;
traces $\{\mathsf{Tr}_i\}$ $(i \in \{1, \ldots, K\})$ with length S;
window length ν, output length m, block index c;
weight limit $w_{\max}$ and top count T.
Output: Top T candidates $\mathrm{Top}_T\{(\mathbf{y}', \mathbf{score}[\mathbf{y}'])\}$.

$\mathcal{C} \leftarrow \{\mathbf{y}' \in \{0,1\}^W : w_H(\mathbf{y}') \le w_{\max}\}$
foreach $\mathbf{y}' \in \mathcal{C}$ **do**
  for $i \leftarrow 1$ **to** K **do**
    Convert $\mathbf{u}_i$ to $m \times \nu$ matrix $\mathbf{U}_i$
    $(\mathbf{h})_i \leftarrow \mathrm{HW}\big(\mathbf{v}_i \oplus \mathbf{U}_i\mathbf{y}'\big)$
    for $k \leftarrow 1$ **to** S **do**
      $(\mathbf{t}_k)_i \leftarrow (\mathsf{Tr}_i)_k$ `// t_k ∈ R^K is the k-th column across traces`
  $\mathbf{score}[\mathbf{y}'] \leftarrow \max_k \left|\mathrm{corr}\big(\mathbf{h}, \mathbf{t}_k\big)\right|$
return $\mathrm{Top}_T\big\{(\mathbf{y}', \mathbf{score}[\mathbf{y}']) : \mathbf{y}' \in \mathcal{C}\big\}$

Algorithm 2: Blockwise DPA Using Algorithm 1

Input: K ciphertext pairs $(\mathbf{u}_i, \mathbf{v}_i)$;
traces $\{\mathsf{Tr}_i\}$ $(i \in \{1, \ldots, K\})$;
projections $\{\mathbf{U}_i\}$;
secret length n, block length ν, output length m, weight limit $w_{\max}$.
Output: Recovered $\hat{\mathbf{y}} \in \{0,1\}^n$.

for $c \leftarrow 0$ **to** $\lceil n/\nu \rceil - 1$ **do**
  $\hat{\mathbf{y}}^{[c,\nu]} \leftarrow$ DPA-Block$\big((\mathbf{u}_i, \mathbf{v}_i), \{\mathsf{Tr}_i\}, \nu, m, c, w_{\max}, 1\big)[0]$ `// invoke Alg. 1; take top-1`
return $\hat{\mathbf{y}} \leftarrow \bigcup_t \hat{\mathbf{y}}^{[t,\nu]}$

Profiled DPA attack approach. In profiling attacks, an attacker first builds a power model using a known-key device. Such models are commonly formulated in two ways, corresponding to generative and discriminative approaches:

1. **Generative modeling**: Model the leakage conditional on an intermediate value. The model synthesizes the estimated leakage for the intermediate value, which is then compared with the measured leakage [SLP05].

2. **Discriminative modeling**: Given a trace, the model predicts the Hamming weight or a related statistic [CRR02].

The evaluation of this paper (in Section 4) will adopt the second approach. We train a compact multilayer perceptron (MLP) on labeled traces to regress the Hamming weight. At attack time, the trained model is applied to each measurement to produce a one-dimensional predicted Hamming-weight trace. Candidate block hypotheses are then scored via Pearson correlation with this derived trace, mirroring the non-profiling procedure. This profiling step transfers information from the labeled training set to the attack phase, typically reducing the number of target queries required.

Sliding-window variant Blockwise DPA is straightforward and robust, but the candidate space grows combinatorially with the Hamming-weight bound, scaling as $\sum_{r=0}^{w_{\max}} \binom{64}{r}$ (see Eq. 4). To accelerate recovery, we apply a sliding-window decomposition: each 64-bit block is covered by three overlapping 32-bit windows with stride 16, namely [0:31], [16:47], and [32:63]. Adjacent windows therefore overlap by half the width. Within each window, we compute the correlation using only the lower sixteen output bits of the projection and treat the remaining bits as unmodeled noise. Window-level hypotheses are ranked independently and then reconciled across overlaps by enforcing bitwise consistency and aggregating scores to evaluate full 64-bit candidates. In practice, this sliding-window scoring recovers blocks with substantially fewer hypotheses than naïve blockwise enumeration while retaining robustness. The domain size then equals

$$|\mathcal{C}_{32,\le w_{\max}}| = \sum_{i=0}^{w_{\max}} \binom{32}{i}.$$

For $w_{\max} = 4$, this yields $|\mathcal{C}_{32,\le 4}| = 41{,}449 \approx 4.14 \times 10^4$. Under the same constraint, The 64-bit domain equals 6.79×10^5 under the same bound. So the 64-bit domain is larger by a factor of about $679{,}121/41{,}449 \approx 16.4$

Local scoring in a sub-block is identical to the blockwise case. Predicted Hamming weights are correlated with traces across $\mathbf{u}$ and $\mathbf{v}$, and the maximum absolute correlation across time serves as the score. High bits outside the sixteen-bit range behave like random noise. Overlap and polynomial stacking average out this noise and stabilize peaks.

Merging across sub-blocks turns local rankings into a coherent 64-bit block decision. The rule is overlap consistency. Two adjacent sub-blocks share a sixteen-bit region. Candidates that disagree on this region are pruned. A beam search maintains the top hypotheses per sub-block and combines them in descending order of total absolute correlation score while discarding inconsistent joins. This favors 64-bit blocks supported by multiple stable local decisions rather than a single dominant sub-block.

Algorithm 3 processes each 64-bit block by sweeping three overlapping sub-blocks of length $\nu = 32$ at offsets $0, 16, 32$ (stride $s = 16$). For each sub-block, we rank candidate patterns by absolute correlation, restrict the search to those with Hamming weight $w_H(\mathbf{y}') \le 4$. We then merge the three candidate lists by enforcing consistency across the shared 16-bit overlaps, using the top $T = 5$ candidates and a beam of width $B = 5$ to speed up merging while keeping the highest-scoring partial assignments in simulations. This yields a compact set of mutually consistent 64-bit hypotheses for the block. According to our simulation and real-device results, varying the stride has little impact on the required number of traces, whereas the choice of B has a noticeably larger effect.

3.4 DPA-Style Attack under Masking

We study correlation-based recovery when the sensitive intermediate is a first-order Boolean share:

$$\mathbf{w} \;=\; \mathbf{v} \oplus (\mathbf{uy}), \qquad \mathbf{w} \;=\; \mathbf{w}^{(0)} \oplus \mathbf{w}^{(1)}.$$

Algorithm 3: Sliding-window Beam DPA per 64-bit block

Input: K ciphertext pairs $(\mathbf{u}_i, \mathbf{v}_i)$;
traces $\{\mathsf{Tr}_i\}$ $(i \in \{1, \ldots, K\})$ with sample length S;
secret length n;
parameters: sub-block length ν, output length m, stride s, beam width W, weight limit $w_{\max}$, top count T.
Output: Recovered $\hat{\mathbf{y}} \in \{0,1\}^n$.

// Slicing convention: $x[a{:}b)$ denotes the subvector $(x_a, \ldots, x_{b-1})$.
$\delta \leftarrow \nu - s$ // with $\nu = 32$, $s = 16$, we have $\delta = 16$
for $b \leftarrow 0$ **to** $\lceil \frac{n}{64} \rceil - 1$ **do**
  $\mathcal{B}_b \leftarrow \varnothing$ // beam over the current 64-bit block
  foreach $t \in \{0, s, 2s\}$ **do**
    // three sub-windows per 64-bit block
    $c \leftarrow 64 \cdot b + t$
    $\mathcal{C}_{b,t} \leftarrow \text{DPA-Block}\big((\mathbf{u}_i, \mathbf{v}_i), \{\mathsf{Tr}_i\}, \nu, m, c, w_{\max}, T\big)$ // invoke Alg. 1; returns $\text{Top}_T(\mathcal{C}_{b,t})$
    if $t = 0$ **then**
      $\mathcal{B}_b \leftarrow \mathcal{C}_{b,0}$
    else
      $\mathcal{B}'_b \leftarrow \varnothing$
      foreach $(y_a, s_a) \in \mathcal{B}_b$ **do**
        foreach $(y_c, s_c) \in \mathcal{C}_{b,t}$ **do**
          if $\delta > 0$ ***and*** $y_a[\nu - \delta : \nu) \neq y_c[0 : \delta)$ **then**
            continue
          $\tilde{y} \leftarrow y_a \,\|\, y_c[\delta : \nu)$
          $S(\tilde{y}) \leftarrow s_a + s_c$ // sum absolute correlations from each sub-window
          if $\tilde{y} \notin \mathcal{B}'_b$ **then**
            insert $(\tilde{y}, S(\tilde{y}))$ into $\mathcal{B}'_b$
          else if $S(\tilde{y})$ *is higher* **then**
            update score in $\mathcal{B}'_b$
      $\mathcal{B}_b \leftarrow \text{Top}_W(\mathcal{B}'_b)$
  if $\mathcal{B}_b = \varnothing$ **then**
    return $\varnothing$ // early exit if beam is empty
  $\hat{\mathbf{y}}^{[b]} \leftarrow \arg\max_{(y,s) \in \mathcal{B}_b} s$
return $\hat{\mathbf{y}} \leftarrow \big\|_b \hat{\mathbf{y}}^{[b]}$

This models the codeword-masking technique in HQC: sampling a random message m' and setting $\mathbf{v}' = \mathbf{v} \oplus m'\mathbf{G}$ gives $\mathbf{w}' = \mathbf{w} \oplus (m'\mathbf{G})$ over $\mathbb{F}_2$ [MNMD25, BMG+25, GSA+25]. Our analysis targets the abstract relation $\mathbf{w} = \mathbf{w}^{(0)} \oplus \mathbf{w}^{(1)}$ and does not rely on decoder behavior.

Sharing hides deterministic equalities on unmasked values, but DPA remains applicable because second-order statistics cancel the random mask on average [DZFL14]. Let $L_0 = w_H(\mathbf{w}^{(0)})$ and $L_1 = w_H(\mathbf{w}^{(1)})$. For a batch of traces with noisy measurements $H0_i$ and $H1_i$, we build the second-order feature

$$Z_i \;=\; \Big(\frac{H0_i - \mu_0}{\sigma_0}\Big)\Big(\frac{H1_i - \mu_1}{\sigma_1}\Big). \tag{5}$$

where μ_0, μ_1 and σ_0, σ_1 are batch means and standard deviations; if a standard deviation is numerically zero, the contribution is set to zero. The predictor remains the Hamming weight of the unmasked intermediate: for candidate $\mathbf{y}^{[c]}$, $\mathbf{U}_i$, and polynomial $\mathbf{v}_i$, use $\mathrm{HW}\big(\mathbf{v}_i \oplus \mathbf{U}_i\mathbf{y}^{[c]}\big)$. We compute correlations between the predictor sequence and Z_i across all time indices.

We use the same method to randomly select polynomials $\mathbf{u}$ across the m indexes. Stacking several independent polynomials spreads each secret bit across many measured configurations; after normalization and fusion, consistent peaks add while independent noise cancels.

The procedure mirrors the unmasked DPA: align traces to the XOR activity of the target slice, mean-center and variance-normalize, precompute $\mathbf{u}$ effects so candidate updates toggle only a few counters, evaluate correlations over the whole time axis, and fuse peaks by z-score averaging. First-order masking lowers SNR, so more traces are typically needed to obtain the same decision margin.

The sliding-window variant transfers directly. Each 64-bit block is decomposed into three 32-bit sub-blocks with a stride 16. In each sub-block, correlation uses the low sixteen output bits of the projection, and the feature is the centered, standardized product of the two share-wise leakages on this block. High bits outside the block act as noise and are averaged out by overlap. Candidate lists from the three sub-blocks are merged by enforcing consistency on the shared sixteen-bit overlaps; a beam search keeps a small set of top hypotheses.

Our methods can be extended to higher-order masking. With $d{+}1$ shares, the product of $d{+}1$ centered share leakages becomes the feature, which is correlated with the predictor derived from the unmasked intermediate; the number of required traces grows with d. Additive sharing implemented inside a Fast Hadamard Transform follows a different protection paradigm [GLG22]; evaluating DPA in that setting requires implementation-specific modeling and is left as future work.

4 Evaluation

This section evaluates the proposed methodology in simulation and on a microcontroller target. We focus on a DPA-style approach instantiated with Pearson correlation, with and without first-order Boolean masking. Our selected $\mathbf{u}$ use locality-preserving Toeplitz blocks, and we analyze both blockwise and sliding-window regimes to balance query complexity and runtime. We report convergence behavior, minimal query budgets, and computational cost, attributing trends to ciphertext diversity and polynomial construction strategy while keeping other settings aligned with Section 3. Unless noted otherwise, traces are acquired on the STM32F4 target as described in Section 4.2, and all remaining experiments and offline computations run on an Apple M1 Ultra workstation (20 CPU cores, 128,GB RAM).

4.1 Simulation Results

We present simulation experiments for HQC-1 to evaluate our method. Synthetic traces follow a Hamming-weight leakage model with additive Gaussian noise, where σ^2 denotes the noise variance. Regarding the SNR, we define it as the ratio between signal power and noise power in our additive leakage model. In the simulated traces, there is only one sampling point, and the Hamming-weight leakage model has a size of 33, from 0 to 32. Other parameters mirror the measurement setup.

Blockwise DPA Attack on unmasked HQC For ciphertext index i, we form $\mathbf{w}_i = \mathbf{v}_i \oplus \mathbf{U}_i\mathbf{y}$ and use $\mathrm{HW}(\mathbf{w}_i)$ as the predictor. The noise abstraction for DPA uses an additive Gaussian model. Each trace sample equals the underlying Hamming weight plus a zero-mean

perturbation with variance σ^2. For each candidate $\mathbf{y}^{[c,64]}$, we compute the maximum absolute Pearson correlation between its predictor sequence and the measured samples along the time axis. A word is declared recovered when the best-scoring candidate is unique with a clear margin. The metric K denotes the smallest number of decapsulation queries that achieves this condition simultaneously for all words.

The projections $\mathbf{U}_i$ are mappings of size $m \times \nu$ that preserve locality as defined in Section 3.2. For each candidate $\mathbf{y}^{[c,\nu]}$, the expensive step is computing $\mathbf{z}_i = \mathbf{U}_i\,\mathbf{y}^{[c,\nu]}$ over $\mathbb{F}_2$. Once $\mathbf{z}_i$ is available, the predictor for trace Tr_i uses the public vector $\mathbf{v}_i$ as $\mathrm{HW}\big(\mathbf{v}_i \oplus \mathbf{z}_i\big)$, which requires one xor on m bits and a popcount and is lightweight compared to forming $\mathbf{U}_i\,\mathbf{y}^{[c,\nu]}$.

Hence, increasing the number of polynomials $\mathbf{u}$ adds one costly matrix–vector evaluation per candidate for each $\mathbf{u}$ (to form $\mathbf{z}_i = \mathbf{U}_i\mathbf{y}^{[c,\nu]}$), which drives an almost linear growth in runtime, whereas adding more polynomials $\mathbf{v}$ mainly adds inexpensive xor and popcount operations on top of the same $\mathbf{z}_i$. Memory usage follows the same trend because per–$\mathbf{u}$ precomputations are stored for each $\mathbf{U}_i$. For concreteness, with $K = 10{,}000$ ciphertext pairs we draw a fresh random $\mathbf{v}$ for every pair and select the polynomial index from several pre-selected $\mathbf{u}$. Under this setup the wall-clock time scales near linearly with number of $\mathbf{u}$: about 1 hour for 10 $\mathbf{u}$, about 2 hours for 100 $\mathbf{u}$, and about 10 hours for 10,000 $\mathbf{u}$.

In the small-noise regime with 10 polynomials $\mathbf{u}$, full recovery is achieved with $K = 12$ traces at $\sigma = 0$ and with $K = 70$ traces at $\sigma = 1$. The correlation margin is large in both cases. In the high-noise regime ($\sigma = 10$), the first configuration uses 10 $\mathbf{u}$ and varies the number of $\mathbf{u}$–public pairs across 3000, 4000, 5000, 6000, and 7000. The corresponding recovery rates over 100 repetitions are 68%, 82%, 87%, 85%, and 86%. The second configuration uses 100 $\mathbf{u}$ with the same trace counts, giving 72%, 88%, 94%, 96%, and 97%. Figure 3 reports the rates versus trace count.

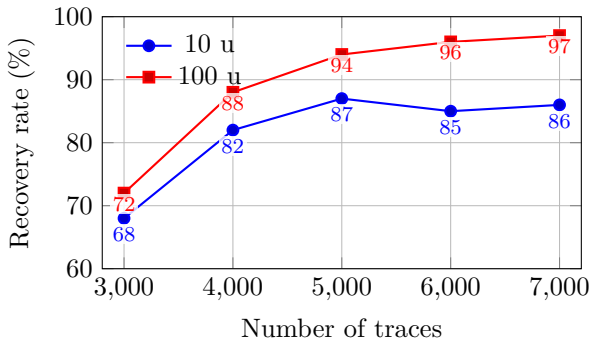


Figure 3: Recovery rate at $\sigma = 10$ versus traces for 10 u and 100 u.

The advantage of additional $\mathbf{u}$ persists when the trace count is very large: with 10 $\mathbf{u}$ and $K = 100{,}000$ traces (or ciphertexts) the recovery rate remains near 95% over 100 repetitions, and with 10,000 $\mathbf{u}$ and $K = 10{,}000$ traces—assigning one public vector to each $\mathbf{u}$—the recovery rate reaches 99%.

Sliding-window DPA Attack on unmasked HQC We evaluate the sliding-window variant under a fixed geometry: $\nu = 32$, $m = 16$, stride 16. For each 64-bit word, we align three blocks at starting indices s, $s + 16$, and $s + 32$. Each block uses ten polynomials $\mathbf{u}$ to instantiate Toeplitz matrices $\mathbf{U}$ that satisfy block closure, so that $\mathbf{U}\mathbf{y}_{[s:s+32]}$ coincides with the modeled band of $(\mathbf{u} * \mathbf{y})_{[s:s+16]}$. Public vectors $\mathbf{v}$ generate the intermediates, and the observation for a given $\mathbf{v}$ equals $\mathrm{HW}\big(\mathbf{v} \oplus \mathbf{U}\mathbf{y}^{\mathrm{true}}_{\mathrm{win}}\big)$ with additive discrete truncated Gaussian noise of standard deviation σ supported on $[0, m]$. The total of K traces is split evenly across the ten projections and three blocks. Candidate enumeration, scoring

by absolute Pearson correlation between concatenated predictions and observations, and overlap-consistent beam merging follow Algorithm 3.

Figure 4 presents, for noise levels $\sigma \in 0, 1, \ldots, 11$, the minimum number of traces required to attain 100% recovery over 100 independent trials. The corresponding counts, ordered by increasing σ, are $K = \{10, 100, 350, 1000, 2500, 4500, 8000, 13000, 18000, 30000, 51000, 65000\}$. The dependence of K on the inverse signal-to-noise ratio is approximately linear, namely $K \propto 1/\mathrm{SNR}$. The sliding-window method remains efficient even under substantial noise: even at $\sigma = 11$ with $K = 65{,}000$, a total 100 runs are completed within seven minutes.

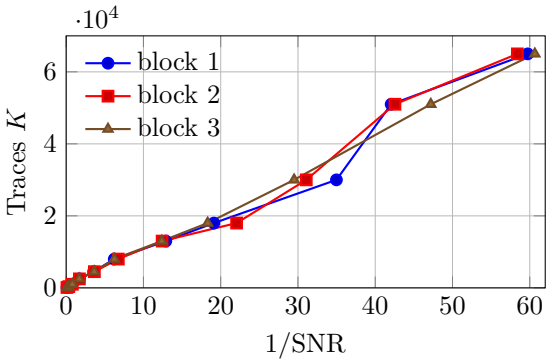


Figure 4: Traces K required to obtain 100% recovery rate vs. 1/SNR for three 32-bit sliding-window.

The sliding-window variant provides only a slight and scenario-dependent improvement in success rate. Its main advantage is computational efficiency: the 16-bit overlaps enable early pruning of inconsistent hypotheses during merging, reducing the overall enumeration workload and accelerating recovery.

DPA-style Attack under first-order codeword masking We evaluate first-order Boolean sharing by writing $\mathbf{w} = \mathbf{w}^{(0)} \oplus \mathbf{w}^{(1)}$ with an independent random mask in the second share (codeword masking). We model the two shares as separate Hamming-weight leakages in simulation and in practice record them at two distinct sample indices in each trace. The simulation uses the second-order feature defined in Eq. (5), setting a term to zero if its standard deviation is numerically zero. For each candidate $\mathbf{y}^{[c]}$, $\mathbf{U}_i$, and public vector $\mathbf{v}_i$, we predict $\mathrm{HW}\big(\mathbf{v}_i \oplus \mathbf{U}_i\mathbf{y}^{[c]}\big)$, correlate it with the feature along time, and score by the peak absolute correlation. Both the blockwise and sliding-window DPA use this feature (Algorithms 1 and 3). All experiments here use ten $\mathbf{u}$ polynomials per block with ciphertexts distributed uniformly.

Table 3: Masked DPA with ten possible values of $\mathbf{u}$ per block: blockwise DPA (Algorithm 1) vs. sliding-window DPA (Algorithm 3).

Method	Noise level σ	Traces for 100% recovery K	Total runtime
Blockwise DPA	0	13 000	160 s
Blockwise DPA	1	21 000	260 s
sliding-window DPA	0	5 000	50 s
sliding-window DPA	1	8 000	96 s

The sliding-window method requires fewer traces and less time than the blockwise approach under identical masking and $\mathbf{u}$ budgets. At $\sigma = 0$ they reach full recovery with about 2.5× fewer traces and about one third of the runtime; at $\sigma = 1$ the gap is similar.

The finer observation granularity and overlap consistency reduce ambiguity and stabilize correlation peaks, keeping computation practical under masking.

4.2 Practical Attack Results

We validate the methodology using power traces from an HQC-1 implementation on an STM32F4 microcontroller. We evaluate DPA attack instantiated with Pearson correlation in both unmasked and first-order Boolean masked settings. Unless otherwise stated, we reuse the parameters from Section 3, namely $(\nu, m) = (64, 32)$ for blockwise analysis and $(32, 16)$ for sliding-window. Trace acquisition is performed on the STM32F4 target, while all post-processing and key-ranking computations are performed offline on the Apple M1 Ultra workstation.

Attack setup We use a ChipWhisperer capture setup with an STM32F4 target and a PicoScope. The HQC implementation[1] follows HQC-1 and is compiled with `-O3`. Traces are sampled at 31.25 MS/s, and for the first 64-bit word we record 200 samples per trace.

Figures 5 and 6 show pointwise SNR and Welch's two-sample t-statistic when modeling leakage as the Hamming weight of the whole 64-bit word versus the lower 32 bits of $\mathbf{w}$. The "low" model exhibits a pronounced burst around the `vect_add` window—higher SNR peaks and a longer region of large $|t|$—while the background is comparable to the 64-bit model, supporting the construction in 3.2. By contrast, the 16-bit and 8-bit HW models show weaker peaks in Figures 5 and smaller $|t|$ excursions in Figure 6, even no better than the 64-bit model, which motivates our choice of $(\nu, m) = (64, 32)$ for the blockwise attack.

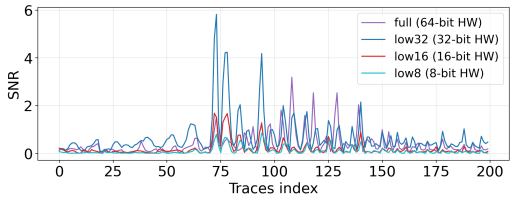


Figure 5: Pointwise SNR.

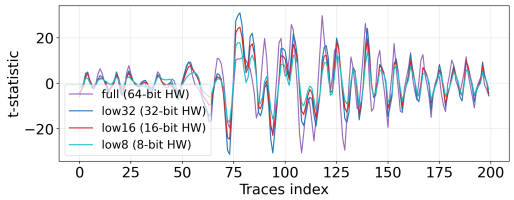


Figure 6: Pointwise Welch's two-sample t-statistic.

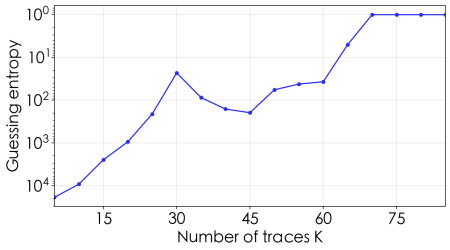
In this experiment, the vector $\mathbf{y}$ is fixed and the pair $(\mathbf{u}, \mathbf{v})$ varies to form $\mathbf{w} = \mathbf{v} \oplus (\mathbf{uy})$. We test leakage on the first word of $\mathbf{w}$. Over $\mathbb{F}_2$, subtraction equals XOR, which matches the targeted implementation site: the wordwise XOR inside `PQCLEAN_HQC128_CLEAN_hqc_pke_decrypt`, specifically `PQCLEAN_HQC128_CLEAN_vect_add` that computes $o[i] = v1[i] \oplus v2[i]$ on 64-bit lanes. Focusing on the first word minimizes confounding from loop control and yields deterministic alignment. Following the simulation study, all DPA acquisitions in this section use five $\mathbf{u}$ per word.

[1] Based on the open-source reference implementation from the PQClean project, available at https://github.com/PQClean/PQClean/tree/master.

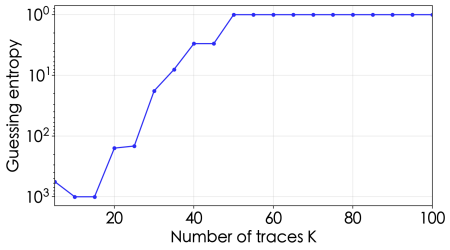
```
1 void PQCLEAN_HQC128_CLEAN_vect_add(uint64_t *o, const uint64_t *v1,
2                                    const uint64_t *v2, size_t size) {
3     for (size_t i = 0; i < size; ++i) {
4         o[i] = v1[i] ^ v2[i]; // attack position
5     }
6 }
```

Figure 7: Attack position in `PQCLEAN_HQC128_CLEAN_vect_add`: the wordwise XOR computing $o[i] = v1[i] \oplus v2[i]$.

Non-Profiling DPA Attack on unmasked HQC Figure 8 visualizes the guessing entropy of the true y-word under the same acquisition and preprocessing for two correlation pipelines. The vertical axis now shows guessing entropy. The blockwise method follows Algorithm 1; the sliding-window method follows Algorithm 3 with three overlapping 32-bit blocks per 64-bit word and overlap-consistent beam merging. In both panels, the guessing entropy improves as the number of traces K increases. In Fig. 8a (a), the blockwise DPA reaches rank 1 around $K = 70$. In Fig. 8b (b), the sliding-window DPA improves monotonically and stabilizes at rank 1 by $K = 50$. These results corroborate that the sliding-window strategy yields a steadier convergence while maintaining the same measurement setup.



(a) Blockwise DPA: guessing entropy of the true y-word vs. number of traces K.



(b) Sliding-window DPA: guessing entropy of the true y-word vs. number of traces K.

Figure 8: Real-device results on the first 64-bit word for non-profiling DPA-style attack

Although the device operates on 32-bit words and the sliding-window method treats the higher 16 bits in each block as noise, it can still require fewer traces than the blockwise attack. This is mainly due to beam-search merging: the 16-bit overlap between adjacent sub-blocks prunes inconsistent candidates and accumulates scores across sub-blocks, which helps suppress random noise. By contrast, blockwise recovery cannot cross-check hypotheses across blocks, although each block may be less noisy. In our setting, we observe that the sliding-window method consistently uses fewer traces, but the advantage depends on the noise level and the windowing configuration.

We further evaluated alternative sliding-window configurations by varying the output length m, the stride s, and the beam width W, and checking whether the attack converges within $K \leq 150$ traces. For each setting, we retain the top $T = 2W$ candidates after scoring each sub-block and merge them using a beam of width W. The results are summarized in Table 4. Overall, our default configuration $m = 32$ and $s = 16$ yields the most favorable behavior, and increasing W tends to accelerate convergence when recovery succeeds. Because the parameter space is large and involves multiple interacting factors, we leave a systematic search for the best configuration to future work.

Profiling DPA Attack on unmasked HQC To complement our non-profiling results and demonstrate the proposed DPA design's applicability in a profiling setting, we instantiate

Table 4: Convergence within $K \leq 150$ traces for different (ν, m, s, W) settings, with $T = 2W$. A cross (×) indicates no convergence.

ν	m	s	$W = 10$	$W = 100$	$W = 1000$
32	16	16	$K = 130$	$K = 65$	$K = 50$
40	16	8	$K = 135$	$K = 140$	$K = 135$
40	16	12	$K = 135$	×	$K = 140$
48	16	8/16	×	×	×
56	16	8	$K = 95$	$K = 95$	$K = 95$
40	32	8	×	×	$K = 130$
40	32	12	×	×	$K = 130$
48	32	8/16	×	×	×
56	32	8	×	×	×

a supervised variant that learns a leakage model from labeled traces collected on the same firmware. The parameters match those of blockwise DPA, with $\nu = 64$ and $m = 32$. Our goal is to quantify how profiling reduces the required trace count K under otherwise identical alignment and scoring procedures. We train a lightweight feedforward MLP to predict the Hamming weight of the m-bit intermediate from short power segments aligned around the XOR computing $\mathbf{w} = \mathbf{v} \oplus (\mathbf{uy})$.

We collect 100,000 labeled traces on the real-world setup and split them into training, validation, and test sets in a 0.8 : 0.1 : 0.1 ratio. All inputs undergo per-trace standardization followed by global feature normalization.

The classifier is a compact MLP with GELU activations and dropout regularization; the final layer is a 33-way softmax. We optimize a joint objective that combines a standard multi-class cross-entropy term with an auxiliary mean-squared error on the predicted expected HW, trained for 100 epochs with mini-batches of size 128 under an adaptive first-order optimizer and a one-cycle learning-rate schedule. Table 5 summarizes the architecture.

Table 5: MLP architecture used for HW prediction of a 32-bit.

Layer	**Output size**	**Activation**	**Dropout**
FC-1	512	GELU	0.2
FC-2	256	GELU	0.2
FC-3	128	GELU	0.2
FC-4	64	GELU	0.2
FC-5	33	Softmax	—

On the test set, the model exactly recovers the correct Hamming weight in 47.37% of cases, is within ±1 of the true value in 91.30%, is within ±2 in 99.45%, and is within ±3 in 100%. We therefore use the predicted Hamming weight as the proxy leakage trace. With the new trace, we next examine how the number of traces K affects the true-y ranking. Figure 9 shows a clear improvement as K increases: the rank on the guessing-entropy plot drops rapidly from 610,567 to 1 and then stabilizes. In particular, the model reaches rank 1 at about $K \approx 20$ and remains at rank 1 for all larger K shown, substantially improving over our best non-profiling result on the same word, which required $K = 50$ traces.

Non-Profiling DPA Attack under first-order masking We reuse the acquisition and preprocessing but switch to the second-order feature in Section 3.4. Candidate generation, scoring, and beam merging remain identical to the unmasked pipelines. We again report a representative case on the first 64-bit word.

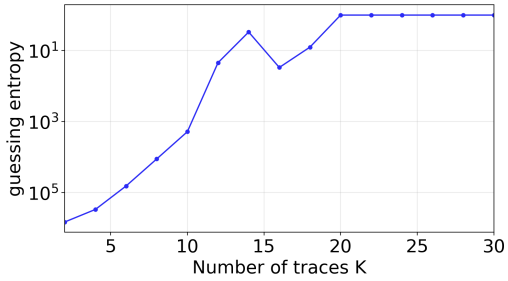
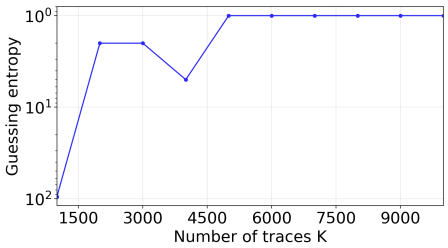
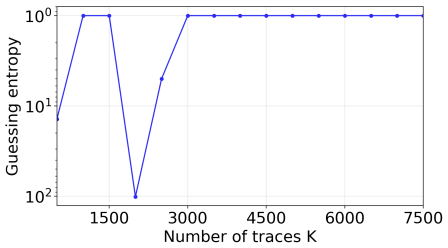


Figure 9: Profiling DPA: guessing entropy of the true y-word vs. number of traces K.

Figure 10 plots the rank of the true y-word against the number of traces K on the guessing entropy axis. In panel (a), the blockwise DPA steadily improves and reaches rank 1 at about $K = 5{,}000$, remaining stable thereafter. In panel (b), the sliding-window DPA with $W = 100$ and $T = 2B$ improves throughout and reaches rank 1 around $K = 3{,}000$, after which it stabilizes; using $W = 1000$ yields a similar threshold, so we report $W = 100$ as the faster setting. These results show that, under first-order masking, both pipelines converge with increasing traces.



(a) Blockwise DPA: guessing entropy of the true y-word vs. number of traces K.



(b) Sliding-window DPA: guessing entropy of the true y-word vs. number of traces K.

Figure 10: First-order masked device experiment on the first 64-bit word. The vertical axis is guessing entropy.

Our profiled method can be extended for the masked implementation. However, profiled second-order attacks typically require more expressive leakage models, such as Gaussian mixture models, to accurately capture the non-Gaussian structure induced by masking and higher-order features. Since this work adopts a simple Gaussian leakage model, we restrict our evaluation of the masked case to the non-profiled scenario.

5 Conclusion and Future Work

We present a practical side-channel framework for HQC decapsulation. It turns cyclic convolution into independent blockwise operations and uses Toeplitz projections to generate linear images that preserve locality and exhibit a strong avalanche effect. Leveraging these dedicated choosing polynomials $\mathbf{u}$, we mount a DPA-style attack instantiated with Pearson correlation that reconstructs the secret block by block. We implement two variants: a blockwise DPA and a sliding-window DPA that reuses overlapping 32-bit sub-blocks and merges them via overlap consistency. On hardware, the blockwise variant recovers a 64-bit block with 70 traces, whereas the sliding-window variant succeeds with 50 traces and reduced runtime.

With first-order codeword masking enabled, a centered second-order feature [DZFL14] restores sufficient signal for recovery: roughly 5,000 traces for the blockwise DPA and 3,000 for sliding-window variant. Together, these results deliver an end-to-end recovery and point to countermeasures that disrupt block locality or suppress low-order leakage at the targeted XOR.

In the meantime, practical implementations can reduce the attack surface by enforcing strict ciphertext sanity checks and by aborting early on decapsulation failures. These measures can prevent malformed ciphertexts from reaching the vulnerable computations, but they are inherently rule-based and cannot anticipate every malformed-ciphertext pattern, and abort-on-failure as a general CCA hardening technique may not be applicable in all settings or may constrain deployment flexibility. Overall, our findings remain meaningful because they expose a concrete and practical side-channel threat against HQC implementations, and they underscore that leakage resilience must be treated as a first-class design and engineering requirement.

Future work. One direction is to optimize the design and scheduling of the polynomials $\mathbf{u}$ to maximize rank, minimize overlap among null spaces, and sharpen hypothesis separability under fixed query budgets. Adaptive, online selection of $\mathbf{u}$ that concentrates queries on the most uncertain 64-bit blocks may further reduce trace counts. An additional avenue for future work is higher-order masking. Our attack targets first-order Boolean masking, whereas higher-order masking for HQC has already been implemented [GSA+25]. Extending our technique would require constructing higher-order, cross-share features and projection schemes that preserve locality while maintaining a usable SNR. A complete treatment of higher-order masking is left to future work.

Acknowledgments

This work was supported by National Key R&D Program of China (2023YFA1009500), the National Natural Science Foundation of China (Grant Nos. 62372273, 62125204 and 92270201), the Key R&D Program of Shandong Province, China (Grant No. 2024ZLGX05), Innovation Program for Quantum Science and Technology (Grant No. 2021ZD0302901 / 2021ZD0302902), the Science and Technology Commission of Shanghai Municipality and the Shanghai Science and Technology Program (Grant Nos. 23511100200, 23511101200 and 23511101202).

References

[BBD+16] Gilles Barthe, Sonia Belaïd, François Dupressoir, Pierre-Alain Fouque, Benjamin Grégoire, Pierre-Yves Strub, and Rébecca Zucchini. Strong non-interference and type-directed higher-order masking. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016*, pages 116–129. ACM, 2016.

[BCO04] Eric Brier, Christophe Clavier, and Francis Olivier. Correlation power analysis with a leakage model. In Marc Joye and Jean-Jacques Quisquater, editors, *Cryptographic Hardware and Embedded Systems - CHES 2004: 6th International Workshop Cambridge, MA, USA, August 11-13, 2004. Proceedings*, volume 3156 of *Lecture Notes in Computer Science*, pages 16–29. Springer, 2004.

[BMG+25] Chloé Baïsse, Antoine Moran, Guillaume Goy, Julien Maillard, Nicolas Aragon, Philippe Gaborit, Maxime Lecomte, and Antoine Loiseau. Secret and shared keys recovery on hamming quasi-cyclic with SASCA. *Des. Codes Cryptogr.*, 93(6):2137–2157, 2025.

[CJRR99] Suresh Chari, Charanjit S. Jutla, Josyula R. Rao, and Pankaj Rohatgi. Towards sound approaches to counteract power-analysis attacks. In Michael J. Wiener, editor, *Advances in Cryptology - CRYPTO '99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings*, volume 1666 of *Lecture Notes in Computer Science*, pages 398–412. Springer, 1999.

[CRR02] Suresh Chari, Josyula R. Rao, and Pankaj Rohatgi. Template attacks. In Burton S. Kaliski Jr., Çetin Kaya Koç, and Christof Paar, editors, *Cryptographic Hardware and Embedded Systems - CHES 2002, 4th International Workshop, Redwood Shores, CA, USA, August 13-15, 2002, Revised Papers*, volume 2523 of *Lecture Notes in Computer Science*, pages 13–28. Springer, 2002.

[DG25a] Haiyue Dong and Qian Guo. Multi-value plaintext-checking and full-decryption oracle-based attacks on HQC from offline templates. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2025(4):254–289, 2025.

[DG25b] Haiyue Dong and Qian Guo. OT-PCA: new key-recovery plaintext-checking oracle based side-channel attacks on HQC with offline templates. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2025(1):251–274, 2025.

[DZFL14] A. Adam Ding, Liwei Zhang, Yunsi Fei, and Pei Luo. A statistical model for higher order DPA on masked devices. In Lejla Batina and Matthew Robshaw, editors, *Cryptographic Hardware and Embedded Systems - CHES 2014 - 16th International Workshop, Busan, South Korea, September 23-26, 2014. Proceedings*, volume 8731 of *Lecture Notes in Computer Science*, pages 147–169. Springer, 2014.

[GAMA+25] Philippe Gaborit, Carlos Aguilar-Melchor, Nicolas Aragon, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Edoardo Persichetti, Gilles Zémor, Jurjen Bos, Arnaud Dion, Jérôme Lacan, Jean-Marc Robert, Pascal Véron, Paulo L. Barreto, Santosh Ghosh, Shay Gueron, Tim Güneysu, Rafael Misoczki, Jan Richter-Brokmann, Nicolas Sendrier, Jean-Pierre Tillich, and Valentin Vasseur. Hamming quasi-cyclic (hqc) specifications. https://pqc-hqc.org/doc/hqc_specifications_2025_08_22.pdf, August 2025. Hamming Quasi-Cyclic (HQC) Speci.

[GBTP08] Benedikt Gierlichs, Lejla Batina, Pim Tuyls, and Bart Preneel. Mutual information analysis: A generic side-channel distinguisher. In *International Workshop on Cryptographic Hardware and Embedded Systems*, pages 426–442. Springer, 2008.

[GLG22] Guillaume Goy, Antoine Loiseau, and Philippe Gaborit. A new key recovery side-channel attack on HQC with chosen ciphertext. In Jung Hee Cheon and Thomas Johansson, editors, *Post-Quantum Cryptography - 13th International Workshop, PQCrypto 2022, Virtual Event, September 28-30, 2022, Proceedings*, volume 13512 of *Lecture Notes in Computer Science*, pages 353–371. Springer, 2022.

[GNNJ23] Qian Guo, Denis Nabokov, Alexander Nilsson, and Thomas Johansson. SCA-LDPC: A code-based framework for key-recovery side-channel attacks on post-quantum encryption schemes. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology - ASIACRYPT 2023 - 29th International Conference on the Theory and Application of Cryptology and Information Security, Guangzhou, China, December 4-8, 2023, Proceedings, Part IV*, volume 14441 of *Lecture Notes in Computer Science*, pages 203–236. Springer, 2023.

[GSA+25] Guillaume Goy, Maxime Spyropoulos, Nicolas Aragon, Philippe Gaborit, Renaud Pacalet, Fabrice Perion, Laurent Sauvage, and David Vigilant. Side-channel sensitivity analysis on HQC: towards a fully masked implementation. *IACR Cryptol. ePrint Arch.*, page 1344, 2025.

[ISW03] Yuval Ishai, Amit Sahai, and David A. Wagner. Private circuits: Securing hardware against probing attacks. In Dan Boneh, editor, *Advances in Cryptology - CRYPTO 2003, 23rd Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 2003, Proceedings*, volume 2729 of *Lecture Notes in Computer Science*, pages 463–481. Springer, 2003.

[KJJ99] Paul C. Kocher, Joshua Jaffe, and Benjamin Jun. Differential power analysis. In Michael J. Wiener, editor, *Advances in Cryptology - CRYPTO '99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings*, volume 1666 of *Lecture Notes in Computer Science*, pages 388–397. Springer, 1999.

[Koc96] Paul C. Kocher. Timing attacks on implementations of diffie-hellman, rsa, dss, and other systems. In Neal Koblitz, editor, *Advances in Cryptology - CRYPTO '96, 16th Annual International Cryptology Conference, Santa Barbara, California, USA, August 18-22, 1996, Proceedings*, volume 1109 of *Lecture Notes in Computer Science*, pages 104–113. Springer, 1996.

[KP00] Çetin Kaya Koç and Christof Paar, editors. *Cryptographic Hardware and Embedded Systems - CHES 2000, Second International Workshop, Worcester, MA, USA, August 17-18, 2000, Proceedings*, volume 1965 of *Lecture Notes in Computer Science*. Springer, 2000.

[MAB+18] Carlos Aguilar Melchor, Nicolas Aragon, Slim Bettaieb, Loıc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Edoardo Persichetti, Gilles Zémor, and IC Bourges. Hamming quasi-cyclic (hqc). *NIST PQC Round*, 2(4):13, 2018.

[MDS99] Thomas S. Messerges, Ezzy A. Dabbish, and Robert H. Sloan. Power analysis attacks of modular exponentiation in smartcards. In Çetin Kaya Koç and Christof Paar, editors, *Cryptographic Hardware and Embedded Systems, First International Workshop, CHES'99, Worcester, MA, USA, August 12-13, 1999, Proceedings*, volume 1717 of *Lecture Notes in Computer Science*, pages 144–157. Springer, 1999.

[Mes00] Thomas S. Messerges. Using second-order power analysis to attack DPA resistant software. In Çetin Kaya Koç and Christof Paar, editors, *Cryptographic Hardware and Embedded Systems - CHES 2000, Second International Workshop, Worcester, MA, USA, August 17-18, 2000, Proceedings*, volume 1965 of *Lecture Notes in Computer Science*, pages 238–251. Springer, 2000.

[MNMD25] Nathan Maillet, Cyrius Nugier, Vincent Migliore, and Jean-Christophe Deneuville. Key recovery from side-channel power analysis attacks on non-simd HQC decryption. In Yael Tauman Kalai and Seny F. Kamara, editors,

Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part V, volume 16004 of *Lecture Notes in Computer Science*, pages 70–102. Springer, 2025.

[Nat24a] National Institute of Standards and Technology. Module-lattice-based digital signature standard. Technical Report 204, Department of Commerce, Washington, D.C., 2024.

[Nat24b] National Institute of Standards and Technology. Module-lattice-based keyencapsulation mechanism standard. Technical Report 203, Department of Commerce, Washington, D.C., 2024.

[Nat24c] National Institute of Standards and Technology. Stateless hash-based digital signature standard. Technical Report 205, Department of Commerce, Washington, D.C., 2024.

[NRR06] Svetla Nikova, Christian Rechberger, and Vincent Rijmen. Threshold implementations against side-channel attacks and glitches. In Peng Ning, Sihan Qing, and Ninghui Li, editors, *Information and Communications Security, 8th International Conference, ICICS 2006, Raleigh, NC, USA, December 4-7, 2006, Proceedings*, volume 4307 of *Lecture Notes in Computer Science*, pages 529–545. Springer, 2006.

[OMHT06] Elisabeth Oswald, Stefan Mangard, Christoph Herbst, and Stefan Tillich. Practical second-order DPA attacks for masked smart card implementations of block ciphers. In David Pointcheval, editor, *Topics in Cryptology - CT-RSA 2006, The Cryptographers' Track at the RSA Conference 2006, San Jose, CA, USA, February 13-17, 2006, Proceedings*, volume 3860 of *Lecture Notes in Computer Science*, pages 192–207. Springer, 2006.

[PRJ+25] Thales B. Paiva, Prasanna Ravi, Dirmanto Jap, Shivam Bhasin, Sayan Das, and Anupam Chattopadhyay. Et tu, brute? side-channel assisted chosen ciphertext attacks using valid ciphertexts on HQC KEM. In Ruben Niederhagen and Markku-Juhani O. Saarinen, editors, *Post-Quantum Cryptography - 16th International Workshop, PQCrypto 2025, Taipei, Taiwan, April 8-10, 2025, Proceedings, Part II*, volume 15578 of *Lecture Notes in Computer Science*, pages 294–321. Springer, 2025.

[SGG24] Robin Leander Schröder, Stefan Gast, and Qian Guo. Divide and surrender: Exploiting variable division instruction timing in HQC key recovery attacks. In Davide Balzarotti and Wenyuan Xu, editors, *33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14-16, 2024*. USENIX Association, 2024.

[SHR+22] Thomas Schamberger, Lukas Holzbaur, Julian Renner, Antonia Wachter-Zeh, and Georg Sigl. A power side-channel attack on the reed-muller reed-solomon version of the HQC cryptosystem. In Jung Hee Cheon and Thomas Johansson, editors, *Post-Quantum Cryptography - 13th International Workshop, PQCrypto 2022, Virtual Event, September 28-30, 2022, Proceedings*, volume 13512 of *Lecture Notes in Computer Science*, pages 327–352. Springer, 2022.

[SLP05] Werner Schindler, Kerstin Lemke, and Christof Paar. A stochastic model for differential side channel cryptanalysis. In Josyula R. Rao and Berk Sunar, editors, *Cryptographic Hardware and Embedded Systems - CHES 2005, 7th International Workshop, Edinburgh, UK, August 29 - September 1, 2005,*

Proceedings, volume 3659 of *Lecture Notes in Computer Science*, pages 30–46. Springer, 2005.

[WJZY23] Weijia Wang, Fanjie Ji, Juelin Zhang, and Yu Yu. Efficient private circuits with precomputation. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2023(2):286–309, 2023.

[WMCS20] Weijia Wang, Pierrick Méaux, Gaëtan Cassiers, and François-Xavier Standaert. Efficient and private computations with code-based masking. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2020(2):128–171, 2020.